UniFECAF · Engenharia de Computação · Sistemas Operacionais

KernelLab

Simulador de Gerenciamento de CPU, Memória e Dispositivos em Sistemas Multitarefa

Relatório técnico — Estudo de caso MedControl Systems

Aluno(a): **Carolina Mendes Correia**

RA: **96043**

Professor(a): **Patricia Ampese**

Outubro de 2026

1. Contextualização do problema

A MedControl Systems opera uma plataforma que monitora equipamentos médicos conectados. O mesmo servidor, com **uma única CPU**, executa ao mesmo tempo monitoramento cardíaco (P1), transmissão de telemetria (P2), geração de relatórios (P3), emissão de alarme crítico (P4), backup (P5) e atualização de dados (P6). Nas últimas semanas foram observados atraso nos alarmes, aumento do tempo de resposta, disputa por CPU, muitas faltas de página, processos de baixa prioridade esperando em excesso, concorrência no acesso a dispositivos e risco de uso indevido de recursos críticos.

Todos esses sintomas têm origem em **gerenciamento de recursos** — exatamente o papel do sistema operacional (TANENBAUM; BOS, 2016): (i) *quem usa a CPU e por quanto tempo* (escalonamento), (ii) *quais páginas ficam na memória física* (memória virtual) e (iii) *quem pode usar cada dispositivo e como evitar acessos simultâneos indevidos* (concorrência e proteção). O KernelLab foi construído para **reproduzir essas decisões e medir o impacto de cada política**, em vez de implementar um SO completo. O fluxo adotado é: entrada dos processos → escalonamento → métricas → memória → dispositivos → comparação.

Dados do cenário (menor número de prioridade = maior prioridade):

Proc	Função	Chegada	Burst	Prior.	Memória	Dispositivo
P1	Monitoramento cardíaco	0	8	1	120 MB	rede
P2	Transmissão de telemetria	1	4	2	80 MB	rede
P3	Geração de relatórios	2	9	4	200 MB	disco
P4	Emissão de alarme crítico	3	2	0	60 MB	audio e rede
P5	Backup do sistema	5	12	5	300 MB	disco
P6	Atualização de dados	6	6	3	150 MB	disco e rede

O processo **P4 (alarme crítico)** é o único com prioridade 0 e burst de apenas 2 ut, mas chega em t=3, quando a CPU já está ocupada por P1. Esse detalhe orienta toda a análise: uma boa política precisa dar à emergência a CPU *imediatamente*, sem punir demais os demais serviços.

2. Estados e ciclo de vida dos processos

Um processo é um programa em execução, com seu contexto (registradores, contador de programa, memória e arquivos abertos). Durante a vida, ele passa por estados gerenciados pelo SO:

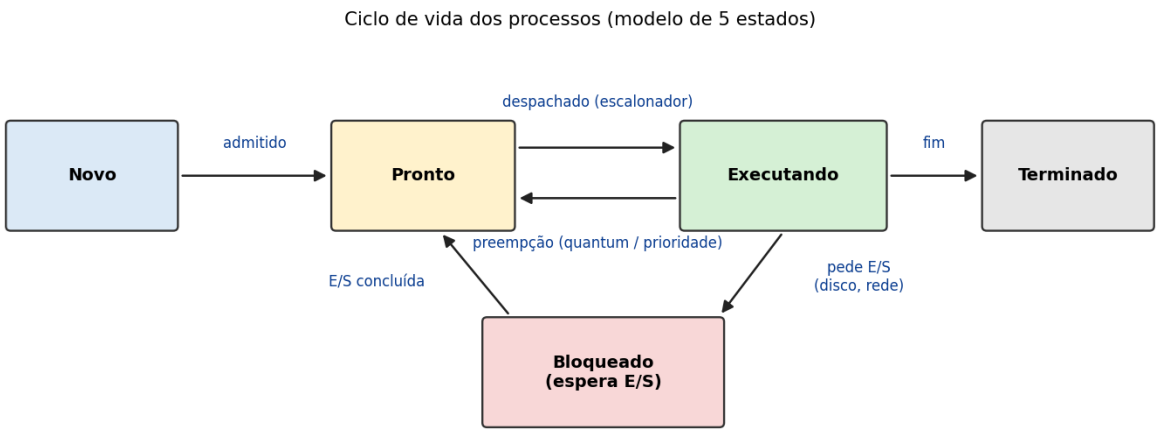


Figura 1 — Modelo de cinco estados e transições.

- **Novo:** processo sendo criado (ex.: o job de backup agendado vira P5).
- **Pronto:** tem tudo para executar e aguarda a CPU na fila de prontos. É aqui que se acumula o *tempo de espera*.
- **Executando:** está de posse da CPU. Em uma CPU só, apenas um processo por vez.
- **Bloqueado:** aguarda um evento (E/S em disco ou rede, liberação de mutex). Não consome CPU.
- **Terminado:** concluiu; recursos devolvidos ao SO.

As transições *Pronto → Executando* (despacho) e *Executando → Pronto* (preempção) são as decisões do escalonador e geram a **troca de contexto**: salvar o estado do processo que sai e carregar o do que entra. A troca é trabalho que não avança nenhum processo — por isso o número de trocas é uma métrica de custo. No KernelLab, os processos do enunciado alternam entre Pronto e Executando; a transição para Bloqueado é representada na Etapa 4, quando P3, P5 e P6 aguardam o disco na fila do mutex.

3. Algoritmos de escalonamento e métricas de desempenho

Métricas (todas em unidades de tempo, ut): **tempo de retorno** = conclusão − chegada; **tempo de espera** = retorno − burst (tempo na fila de prontos); **tempo de resposta** = primeira execução − chegada. Além delas: **trocas de contexto** (custo) e **justiça**. Para justiça usei o **índice de Jain** (JAIN, 1991) sobre x_i = $burst_i$ / $retorno_i$, a fração do tempo no sistema em que cada processo realmente usou a CPU: $J = (\Sigma x)^2 / (n \cdot \Sigma x^2)$, de 1/n (injusto) a 1 (perfeitamente justo). **Starvation** (inanição) é o adiamento indefinido de um processo pronto; **atraso excessivo** é o adiamento longo, porém limitado — o simulador sinaliza espera > 15 ut ou espera/burst ≥ 2.

3.1 FCFS — First Come, First Served

Fila simples por ordem de chegada, sem preempção: o processo roda até terminar. É fácil de implementar e livre de starvation, mas sofre o **efeito comboio**: processos curtos esperam atrás de longos. Aqui, o alarme P4 (2 ut) aguarda P1, P2 e P3 e só começa em t=21 — **18 ut de espera**.

```
P1 [0-8] | P2 [8-12] | P3 [12-21] | P4 [21-23] | P5 [23-35] | P6 [35-41]
```

Proc	Chegada	Burst	Início	Fim	Espera	Retorno	Resposta
P1	0	8	0	8	0	8	0
P2	1	4	8	12	7	11	7
P3	2	9	12	21	10	19	10
P4	3	2	21	23	18	20	18
P5	5	12	23	35	18	30	18
P6	6	6	35	41	29	35	29
Média					13,67	20,50	13,67

3.2 SJF — Shortest Job First

Entre os prontos, executa o de menor burst (versão não preemptiva). Entre algoritmos não preemptivos, é o que **minimiza o tempo médio de espera**: colocar um job curto antes de um longo reduz a espera de muitos e aumenta a de poucos (TANENBAUM; BOS, 2016). Exige conhecer — ou estimar — o burst, e **pode causar starvation** de jobs longos se jobs curtos continuarem chegando. Aqui, P5 (12 ut) é o último e espera 24 ut.

`P1 [0-8] | P4 [8-10] | P2 [10-14] | P6 [14-20] | P3 [20-29] | P5 [29-41]`

Proc	Chegada	Burst	Início	Fim	Espera	Retorno	Resposta
P1	0	8	0	8	0	8	0
P2	1	4	10	14	9	13	9
P3	2	9	20	29	18	27	18
P4	3	2	8	10	5	7	5
P5	5	12	29	41	24	36	24
P6	6	6	14	20	8	14	8
Média					10,67	17,50	10,67

3.3 Round Robin (quantum = 3)

Cada processo recebe uma fatia de 3 ut; se não termina, volta ao fim da fila (preempção por tempo). Garante que todos avancem e que a espera por rodada seja limitada a (n−1)·q — logo **não há starvation** — e oferece bom tempo de resposta. O preço é o **maior número de trocas de contexto** e retornos maiores, pois todos terminam tarde. O quantum é um compromisso: muito grande degenera em FCFS (o teste automatizado confirma isso); muito pequeno gasta a CPU em trocas.

`P1 [0-3] | P2 [3-6] | P3 [6-9] | P4 [9-11] | P1 [11-14] | P5 [14-17] | P6 [17-20] | P2 [20-21] | P3 [21-24] | P1 [24-26] | P5 [26-29] | P6 [29-32] | P3 [32-35] | P5 [35-41]`

Proc	Chegada	Burst	Início	Fim	Espera	Retorno	Resposta
P1	0	8	0	26	18	26	0
P2	1	4	3	21	16	20	2
P3	2	9	6	35	24	33	4
P4	3	2	9	11	6	8	6
P5	5	12	14	41	24	36	9
P6	6	6	17	32	20	26	11
Média					18,00	24,83	5,33

3.4 Escalonamento por prioridade

Executa o processo pronto de maior prioridade (menor número). Na versão **não preemptiva**, um alarme que chega durante uma execução precisa esperar o término dela (P4 espera P1 terminar em t=8: 5 ut). Na versão **preemptiva**, o processo mais prioritário que chega toma a CPU imediatamente. O risco é a **starvation** dos de baixa prioridade; a solução clássica é o **aging**, que aumenta a prioridade de quem espera. Implementei as duas versões e o aging, com piso (*aging_floor*=1) para que a classe 0 permaneça reservada a processos críticos.

Não preemptivo: `P1 [0-8] | P4 [8-10] | P2 [10-14] | P6 [14-20] | P3 [20-29] | P5 [29-41]`

Proc	Chegada	Burst	Início	Fim	Espera	Retorno	Resposta
P1	0	8	0	8	0	8	0
P2	1	4	10	14	9	13	9
P3	2	9	20	29	18	27	18
P4	3	2	8	10	5	7	5
P5	5	12	29	41	24	36	24
P6	6	6	14	20	8	14	8
Média					10,67	17,50	10,67

Preemptivo: `P1 [0-3] | P4 [3-5] | P1 [5-10] | P2 [10-14] | P6 [14-20] | P3 [20-29] | P5 [29-41]`

Proc	Chegada	Burst	Início	Fim	Espera	Retorno	Resposta
P1	0	8	0	10	2	10	0
P2	1	4	10	14	9	13	9
P3	2	9	20	29	18	27	18
P4	3	2	3	5	0	2	0
P5	5	12	29	41	24	36	24
P6	6	6	14	20	8	14	8
Média					10,17	17,00	9,83

Observação: SJF e prioridade não preemptiva produzem o mesmo resultado porque, após t=8, a ordem por burst (2, 4, 6, 9, 12) coincide com a ordem por prioridade (0, 2, 3, 4, 5). É uma coincidência dos dados, não equivalência dos algoritmos; com outros valores (editando o JSON) eles divergem.

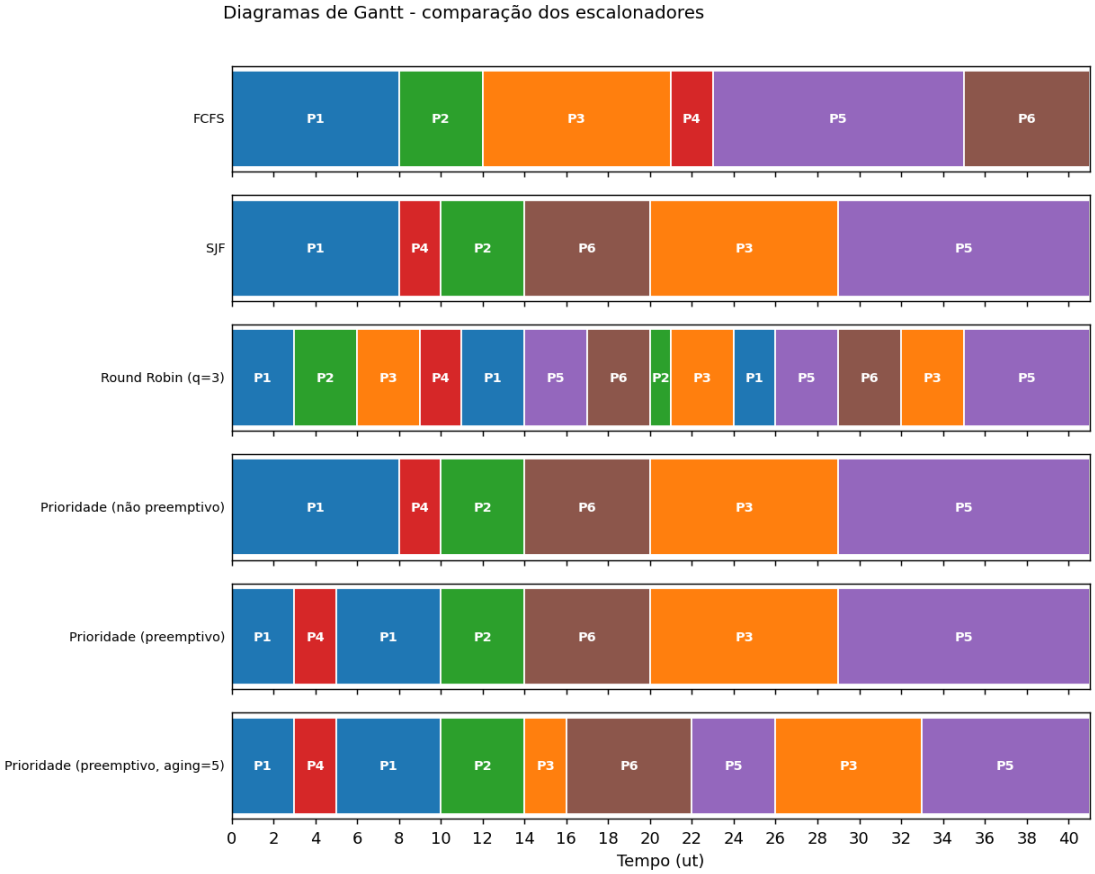


Figura 2 — Diagramas de Gantt dos algoritmos (gerado pelo simulador).

4. Comparação dos escalonadores

Algoritmo	Espera	Retorno	Resposta	Trocas	P4 espera	Jain
FCFS	13,67	20,50	13,67	5	18	0,674
SJF	10,67	17,50	10,67	5	5	0,761
Round Robin (q=3)	18,00	24,83	5,33	13	6	0,972
Prioridade (não preemptivo)	10,67	17,50	10,67	5	5	0,761
Prioridade (preemptivo)	10,17	17,00	9,83	6	0	0,799
Prioridade (preemptivo, aging=5)	11,17	18,00	8,00	8	0	0,777

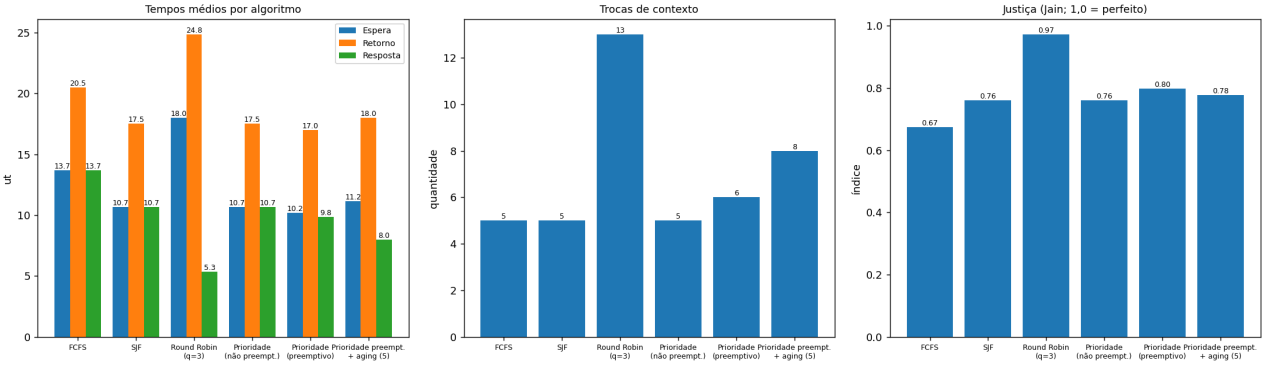


Figura 3 — Tempos médios, trocas de contexto e justiça.

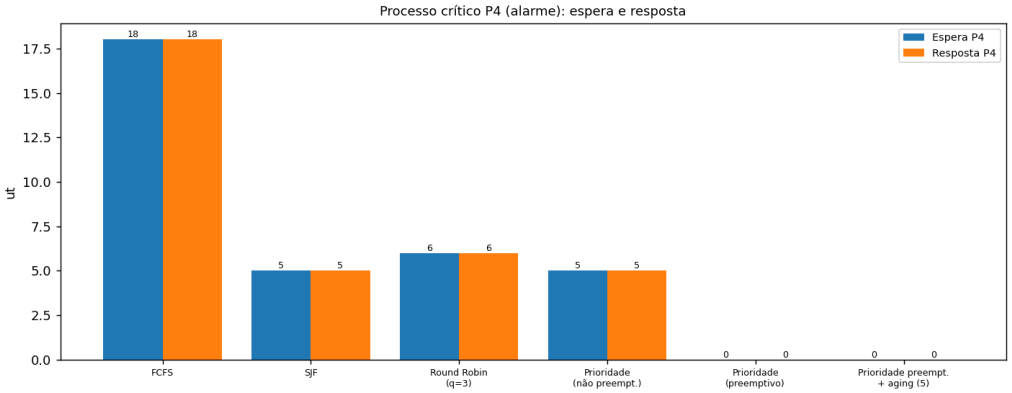


Figura 4 — Tratamento do processo crítico P4.

- **Menor espera média:** entre os quatro obrigatórios, SJF e Prioridade não preemptiva empatam (10,67 ut). Considerando a variante preemptiva, a menor é a **Prioridade preemptiva (10,17 ut)**.
- **Melhor tempo de resposta: Round Robin** (5,33 ut), pois todo processo toca a CPU logo na primeira rodada.
- **Alarme (P4):** a **Prioridade preemptiva** é a mais eficiente (espera e resposta 0 ut). SJF e prioridade não preemptiva fazem o alarme esperar 5 ut; o Round Robin, 6 ut; o FCFS, 18 ut.
- **Maior risco de starvation: prioridade** (e SJF), que concentram o atraso em P5 (24 ut) e P3 (18 ut). No cenário base todos terminam, mas no **teste de estresse** (chegada contínua de processos prioritários) o processo de baixa prioridade espera 23 ut sem aging e 12 ut com aging (intervalo 3). FCFS e RR têm espera limitada.
- **CPU mais justa: Round Robin** (Jain = 0,97), contra 0,67 do FCFS.

Qual é mais adequado para a MedControl? Nenhuma métrica isolada decide. O RR é o mais justo e rápido para responder, mas é o pior em espera média (18,00 ut) e ainda atrasa o alarme

em 6 ut. FCFS é inaceitável para o alarme (18 ut). SJF/prioridade não preemptiva são bons na média, mas o alarme ainda espera 5 ut. A **prioridade preemptiva** combina a menor espera média (10,17 ut), o alarme sem espera e apenas 6 trocas de contexto. Seu ponto fraco — starvation de P5 e P3 — é tratado com **aging**, que no cenário base tem custo (espera média 10,17 → 11,17 ut; trocas 6 → 8) e no estresse evita a inanição. A recomendação é, portanto, **prioridade preemptiva com aging calibrado** (e Round Robin apenas entre processos de mesma prioridade).

5. Memória virtual e algoritmos de substituição de páginas

A **memória virtual** permite que a soma dos espaços de endereçamento dos processos exceda a RAM: o espaço virtual é dividido em **páginas**, a RAM em **frames** do mesmo tamanho, e a **tabela de páginas** (acelerada pela TLB) traduz endereços virtuais em físicos. Se a página referenciada não está na RAM, ocorre uma **falta de página (page fault)**: o SO bloqueia o processo, busca a página no disco, escolhe uma **vítima** se não houver frame livre e reinicia a instrução. Uma falta custa ordens de grandeza mais que um acesso à memória; o sistema só funciona bem por causa da **localidade de referência**. Se faltam frames para o conjunto de trabalho, o sistema entra em *thrashing* (TANENBAUM; BOS, 2016).

No cenário, os seis processos somam 910 MB; se a RAM do servidor for menor, haverá paginação, e as faltas atrasam justamente os processos críticos. Isso sugere duas decisões: usar um bom algoritmo de substituição e **manter as páginas de P4 (e do código do escalonador) residentes** na RAM (*page pinning*/*mlock*), para que o alarme nunca espere o disco.

Algoritmos: FIFO remove a página mais antiga na memória — simples, mas ignora o uso e pode remover páginas muito acessadas; sofre da **anomalia de Belady** (mais frames podem gerar mais faltas). **LRU** remove a página menos recentemente usada, aproximando o futuro pelo passado recente (localidade temporal); é um algoritmo de pilha, sem anomalia, mas exige hardware ou estruturas custosas, motivo pelo qual se usam aproximações (relógio/NRU). **Optimal** (Belady) remove a página cujo próximo uso está mais distante; é o *limite inferior* de faltas, impossível na prática (exige conhecer o futuro) e usado como referência.

Resultados (4 frames; referências: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, 2, 1, 5, 3, 2; 17 acessos):

Algoritmo	Page hits	Page faults	Taxa de faltas
FIFO	5	12	70,6%
LRU	7	10	58,8%
Optimal	10	7	41,2%

FIFO — estado dos frames após cada referência (F1–F4):

#	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Ref	1	2	3	4	1	2	5	1	2	3	4	5	2	1	5	3	2
F1	1	1	1	1	1	1	5	5	5	5	4	4	4	4	4	4	4
F2	-	2	2	2	2	2	2	1	1	1	1	5	5	5	5	5	5
F3	-	-	3	3	3	3	3	3	2	2	2	2	2	1	1	1	1
F4	-	-	-	4	4	4	4	4	4	3	3	3	3	3	3	3	2
Res.	F	F	F	F	H	H	F	F	F	F	F	F	H	F	H	H	F

LRU — estado dos frames após cada referência (F1–F4):

#	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Ref	1	2	3	4	1	2	5	1	2	3	4	5	2	1	5	3	2
F1	1	1	1	1	1	1	1	1	1	1	1	5	5	5	5	5	5
F2	-	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
F3	-	-	3	3	3	3	5	5	5	5	4	4	4	4	4	3	3
F4	-	-	-	4	4	4	4	4	4	3	3	3	3	1	1	1	1
Res.	F	F	F	F	H	H	F	H	H	F	F	F	H	F	H	F	H

Optimal — estado dos frames após cada referência (F1–F4):

#	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Ref	1	2	3	4	1	2	5	1	2	3	4	5	2	1	5	3	2
F1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	3	3
F2	-	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
F3	-	-	3	3	3	3	3	3	3	3	4	4	4	4	4	4	4
F4	-	-	-	4	4	4	5	5	5	5	5	5	5	5	5	5	5
Res.	F	F	F	F	H	H	F	H	H	H	F	H	H	H	H	F	H

H = page hit; F = page fault. A tabela completa com a página removida em cada falta está em docs/resultados.md.

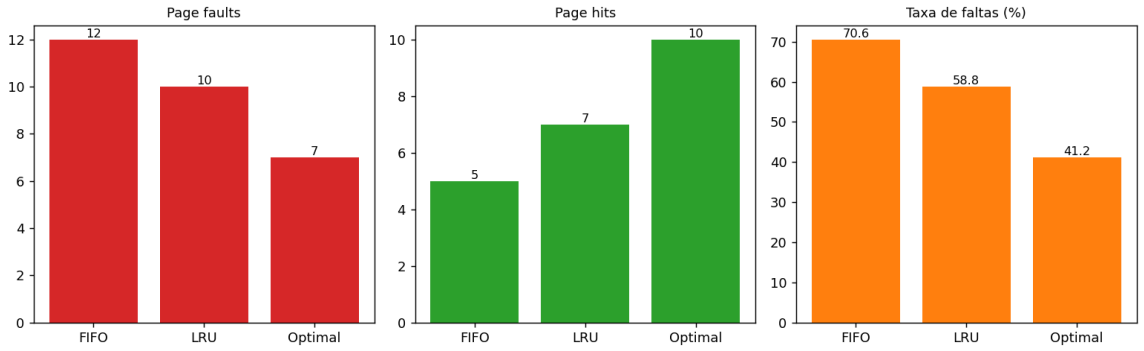


Figura 5 — Hits, faltas e taxa de faltas.

Melhor resultado: o **Optimal** (7 faltas) é o melhor, como esperado, por ser a referência teórica; entre os implementáveis, o **LRU** (10) supera o FIFO (12). **Justificativa:** na referência 5 (7º acesso), os quatro frames guardam {1,2,3,4}. O FIFO remove a página 1, apenas por ser a mais antiga, embora 1 e 2 sejam reutilizadas nos dois acessos seguintes; isso gera uma cascata de quatro faltas consecutivas (acessos 8 a 11). O LRU remove a 3 (usada há mais tempo), mantendo 1 e 2 e ganhando dois hits. O Optimal remove a 4 (cujo próximo uso é o mais distante, no 11º acesso) e ganha três hits seguidos. A diferença vem de quanto cada política consegue explorar a localidade: FIFO ignora-a, LRU a infere do passado, Optimal a conhece do futuro.

Sensibilidade ao número de frames (o simulador permite alterar frames e sequência):

Frames	1	2	3	4	5	6
FIFO	17	17	14	12	5	5
LRU	17	17	14	10	5	5
Optimal	17	13	9	7	5	5

Com 5 frames só ocorrem as faltas compulsórias (5 páginas distintas). Para a sequência clássica 1,2,3,4,1,2,5,1,2,3,4,5, o FIFO **piora** de 9 faltas (3 frames) para 10 (4 frames) — a anomalia de Belady, verificada em teste automatizado; LRU e Optimal não sofrem esse efeito.

6. Concorrência, mutex e semáforos

Quando vários processos ou threads acessam um recurso compartilhado, a correção pode depender da ordem de intercalação: é uma **condição de corrida**. O trecho que acessa o recurso é a **região crítica** e precisa de **exclusão mútua** (TANENBAUM; BOS, 2016). Na MedControl, P3, P5 e P6 gravam no disco; P1, P2, P4 e P6 usam a rede; todos gravam no log. Sem controle, dois processos podem intercalar escritas e corromper arquivos ou contadores.

Mutex é uma trava binária: quem a adquire entra na região crítica; os demais bloqueiam (estado *Bloqueado* do ciclo de vida) até a liberação. **Semáforo** é um contador com operações atômicas *down/up*; com valor inicial N, permite até N usuários simultâneos (ex.: um pool de conexões de rede), e com N = 1 equivale a um mutex. Cuidados: **deadlock** (P6 usa disco e rede; se P6 pegar disco→rede e outro processo rede→disco, podem travar; a solução é impor uma ordem global de aquisição) e **inversão de prioridade** (um processo de baixa prioridade segura o mutex de que o alarme precisa; mitigada por herança de prioridade).

Implementação no KernelLab: a classe *Device* usa *threading.Semaphore(capacity)*; capacity=1 (disco, áudio, log) é um mutex e capacity=2 modela a rede com duas conexões. O acesso **exclusivo** captura todas as vagas, e uma trava de entrada impede que outros furem a fila. Resultados da demonstração: **sem mutex**, 5 threads incrementando 200 vezes um contador compartilhado (simulando o contador do arquivo de log) terminaram bem abaixo de 1000 por causa de atualizações perdidas (o valor exato varia em cada execução); **com mutex**, o resultado foi exatamente 1000. Com threads de P3, P5 e P6 disputando o disco, os intervalos de uso ficaram sequenciais, com **0 sobreposições**. Na simulação determinística (tempo simulado), P3 usa o disco de 2 a 6, P5 espera 3 ut (6 a 11) e P6 espera 7 ut (11 a 14).

7. Segurança no acesso aos dispositivos

O risco apontado pela MedControl — processos não autorizados usando recursos críticos — é tratado com **controle de acesso** e o **princípio do menor privilégio**: cada processo recebe apenas as permissões de que precisa. A **matriz de controle de acesso** (domínios de proteção × objetos) é verificada pelo SO a cada operação; permissão ausente gera *AccessDenied*. Permissões modeladas: R (ler), W (gravar), X (executar), U (utilizar o dispositivo) e E (solicitar acesso exclusivo).

Proc	disco	rede	áudio	log
P1	-	RWU	-	W
P2	-	RWU	-	W
P3	RWU	-	-	W
P4	-	WXUE	WXUE	W
P5	RWXUE	-	-	W
P6	RWU	RWU	-	W

- **Justificativa:** P1 e P2 só usam rede (sem disco nem áudio); P3 só disco; P5 só disco; P6 disco e rede; P4 áudio e rede.
- **Acesso exclusivo (E)** apenas onde é indispensável: P4 em áudio/rede (o alarme não pode ser interrompido) e P5 em disco (consistência do backup).
- **Log somente de escrita (W):** nenhum processo lê o log, o que dificulta adulteração e vazamento (modelo *append-only*).
- **Negações verificadas:** P1 → disco (W), P3 → rede (U) e P2 → áudio (U) são negados; P1 pedindo o disco é recusado antes de entrar na fila.

Limites: a matriz é estática e baseada em PID, sem autenticação, auditoria, nem isolamento por usuário; em produção seriam necessários ACLs/capabilities, separação de privilégios e registros de auditoria.

8. Relação entre conceitos e decisões implementadas

Sintoma na MedControl	Conceito	Decisão no KernelLab
Atraso do alarme	Preempção e prioridade	Prioridade preemptiva; P4 com espera 0 ut
Processos de baixa prioridade esperando	Starvation e aging	Aging com piso; teste de estresse (23 → 12 ut)
Aumento do tempo de resposta	Fatia de tempo	RR mede resposta (5,33 ut) e o custo em trocas (13)
Disputa pela CPU	Escalonamento e métricas	4 algoritmos comparados por espera, retorno, resposta e Jain
Muitas faltas de página	Memória virtual, localidade	LRU (10) em vez de FIFO (12); fixar páginas de P4
Concorrência em dispositivos	Região crítica, mutex/semáforo	Device com semáforo, fila FIFO e modo exclusivo
Processos não autorizados	Proteção, menor privilégio	Matriz R/W/X/U/E com AccessDenied

9. Recomendação final e limitações

Recomendação: adotar **escalonamento por prioridade preemptivo com aging**; usar **LRU** (ou aproximação como o algoritmo do relógio) na memória, mantendo residentes as páginas do alarme; proteger disco, rede, áudio e log com **matriz de menor privilégio** e **mutex/semáforos**, com acesso exclusivo restrito a P4 e P5 e ordem global de aquisição para evitar deadlock.

Limitações que permanecem: modelo de uma CPU com bursts conhecidos (SJF e prioridade dependem de estimativas em sistemas reais); não há custo explícito da troca de contexto nem E/S bloqueante; a inversão de prioridade não é tratada (exigiria herança de prioridade); não há garantia de tempo real rígido (EDF/RMS seriam mais adequados a prazos estritos de alarme); sequência de páginas única e curta (17 acessos), de modo que as conclusões sobre memória são ilustrativas; a matriz de acesso é estática. Melhorias: SRTF, MLFQ, algoritmo do relógio, múltiplas CPUs, E/S com bloqueio, interface web.